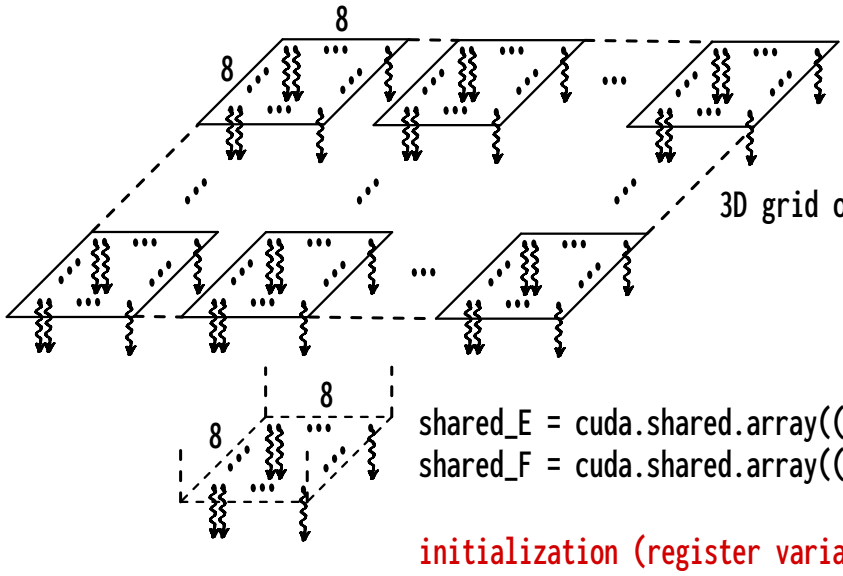
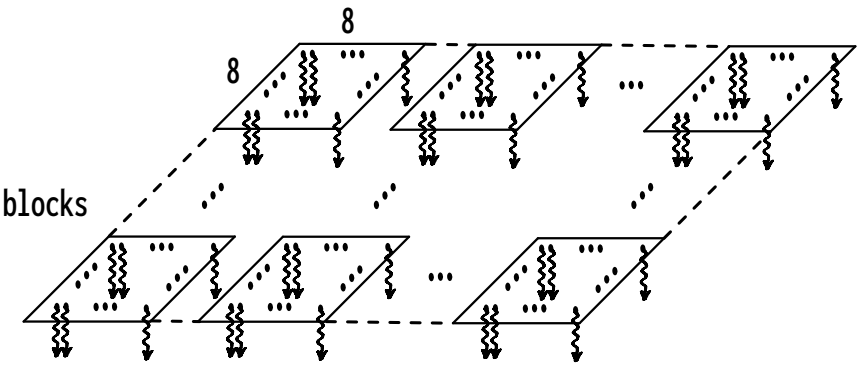
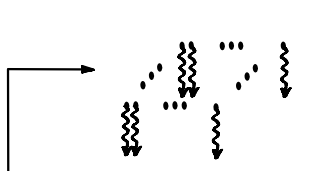
```
@cuda.jit(void(float32[:, :], float32[:, :], float32, float32[:, :]))
matmuldiag_numba_cuda_job_float32[bpg, tpb](E4, F4, factor, G4)
# tile_size = 8
# bpg_y = (R + tile_size - 1) // tile_size, bpg_x = (T + tile_size - 1) //tile_size
# bpg = (bpg_x, bpg_y, 4), tpb = (tile_size, tile_size)
```



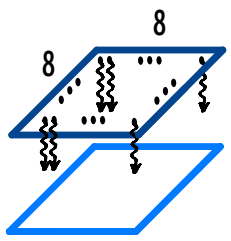
3D grid of thread blocks



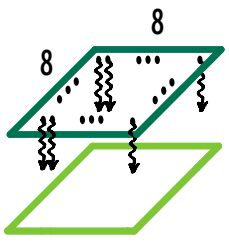

```
shared_E = cuda.shared.array((8, 8), dtype=float32)
shared_F = cuda.shared.array((8, 8), dtype=float32)

initialization (register variables of each thread):
tile_size = cuda.blockDim.x
R4, T = G4.shape
S4 = F4.shape[0]
R, S = R4 >> 2, S4 >> 2
bx, by, bz = cuda.blockIdx.x, cuda.blockIdx.y, cuda.blockIdx.z
tx, ty = cuda.threadIdx.x, cuda.threadIdx.y
row, col = by * tile_size + ty, bx * tile_size + tx
tmp = float32(0.0)
row_bz_R, ty_bz_S = row + bz * R, ty + bz * S
```



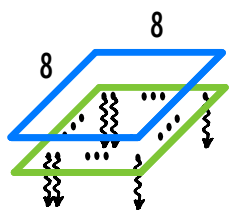

```
loop over tiles of inner products (index incremented by tile_size):
for k in range(0, S, tile_size):
```



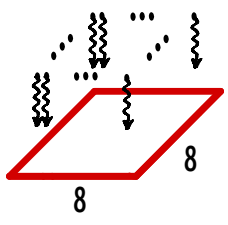

```
    copying current tile of E4 to shared_E:
    shared_E[ty, tx] = E4[row_bz_R, k + tx] if row < R and k + tx < S else float32(0.0)
```




```
    copying current tile of F4 to shared_F:
    shared_F[ty, tx] = F4[k + ty_bz_S, col] if k + ty < S and col < T else float32(0.0)
```




```
    cuda.syncthreads()

    updating inner product tmp of each thread:
    for s in range(tile_size):
        tmp += shared_E[ty, s] * shared_F[s, tx]

    cuda.syncthreads()
```




```
copying inner products to output array G4:
if row < R and col < T:
    G4[row_bz_R, col] = factor * tmp
```